Frontend y Backend

Los términos `frontend` y `backend` van muy de la mano con los términos de *cliente* y *servidor*.

En esencia, el `frontend` es el cliente web y el `backend` es el servidor web.

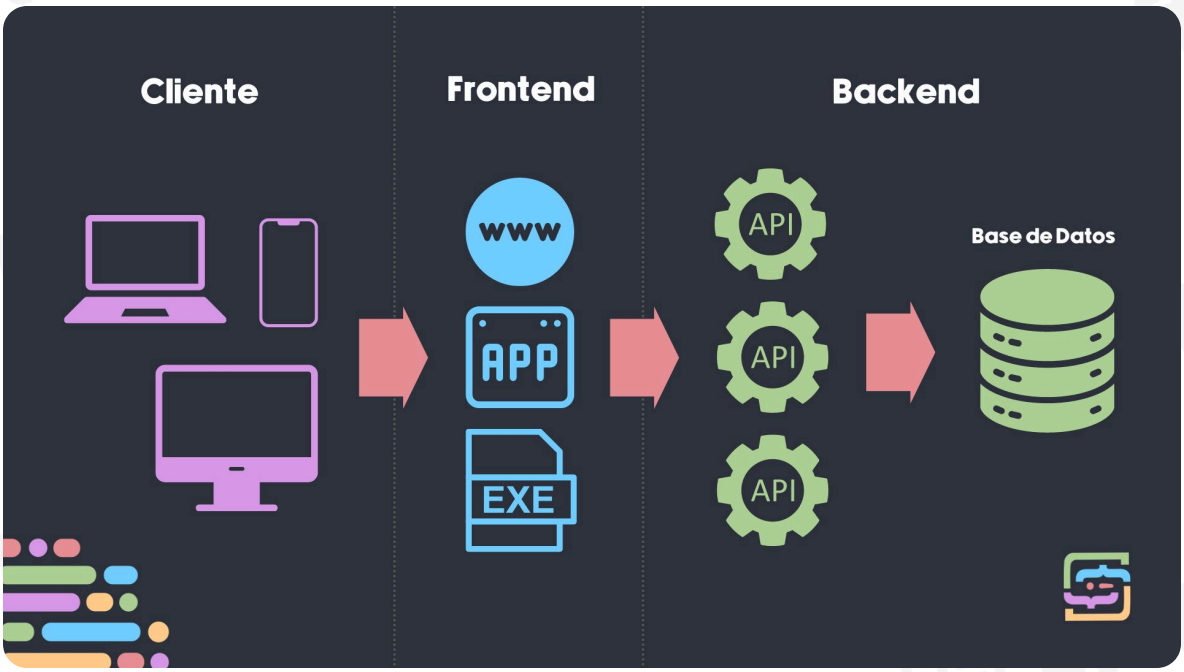
El `frontend` se refiere a lo visual, y el `backend` a todo lo que no podemos ver.

Cuando usamos apps como la de Instagram, TikTok o Youtube, estas se conectan al mismo `backend` que las páginas web (de Instagram, TikTok y Youtube).

Es por esto que podemos ver el mismo contenido tanto en las apps como en la página web.

Son clientes distintos que se conectan al mismo servidor.

Por ejemplo, cuando hago click al botón de like en Facebook, el frontend le envía una solicitud al backend para guardar ese like.



Un `frontend` puede ser cualquier sistema desarrollado usando cualquier lenguaje o tecnología.

Por ejemplo, el Gestor de Finanzas Personales podría ser un frontend si se comunicara con un `backend`.

Normalmente existen 3 tipos:

Los de programas (como programas de Windows).

Los de aplicaciones (móviles o de TV).

Los web.

De ahora en adelante, cada vez que hablemos de `frontend`, estaremos hablando de `frontend` **web**.

El `backend` puede ser cualquier `API` desarrollada usando cualquier lenguaje o tecnología.

Se encarga de la lógica del sistema (*lógica de negocio*), así como de de guardar, actualizar y borrar datos de la base de datos.

Normalmente existen 3 tipos de `API` (las que vimos en APIs):

Rest

SOAP

GraphQL

Sin embargo, el `backend` no se limita solo al API.

También incluye tareas asíncronas, la base de datos, el caché, modelos de pub-sub, etc.

Frontend (web)

El único lenguaje de programación que puede ser usado para desarrollar un `frontend` **web** es `Javascript`.

Esto debido a que los navegadores web son, en esencia, *interpretes de* `Javascript`.

Existen algunas tecnologías que permiten desarrollar `frontend` web en otros lenguajes, pero lo que hacen es *transpilar* esos lenguajes a `Javascript`.

Hay otros dos lenguajes importantes en **cualquier** `frontend` web: `HTML` y `CSS`.

Sin embargo, **estos no son lenguajes de programación.**

HTML

Es un lenguaje de *etiquetas* encargado de definir la estructura de la página.

> Con él definimos donde hay botones, donde hay texto, donde hay imágenes, etc.

`HTML` es muy sencillo y al grano.

> Es como escribir un ensayo en Word.

CSS

Es un lenguaje de *estilo* encargado de definir el estilo visual de la página.

> Con él definimos cómo se ven los botones, de qué tamaño es en texto, dónde se posicionan las imágenes, etc.

Existen varios frameworks de `CSS` que tienen estilos predefinidos, como `Bootstrap`, `Tailwind` o `MaterialUI`.

Javascript

Es **el lenguaje de programación** encargado de definir cómo funcionan las interacciones del usuario de la página.

> Con él definimos qué hacen los botones, cuando se muestra o se enconde un texto, cómo agregar o quitar imágenes, etc.

Existen varios frameworks de `JS` que tienen distintas maneras de facilitar la programación de esas interacciones, como `React`, `Angular`, `VueJS` o `Svelte`.

> Todos tienen similitudes, pero funcionan de maneras totalmente distintas.
>
> Hay muchas maneras de hacer exactamente lo mismo.

Tambien existe `Typescript`, un superset *typeado* (recordemos el concepto en Typeados y dinámicos) de `JS` que se puede usar de manera intercambiable con `JS`.

Backend

El `backend` puede ser desarrollado en casi cualquier lenguaje y tecnología.

La comunicación con el `frontend` se da por medio de `APIs`.

`JS` (el frontend) llama a esos `APIs` para solicitar o modificar información.

Estos `APIs` se conectan con la *base de datos* y con otros `APIs` para guardar esa información.

El `backend` es un área mucho más madura que el `frontend`.

Ya existen las "*mejores maneras*" de hacer casi cualquier cosa.

La mayoría de frameworks y tecnologías de `backend` funcionan exactamente igual con algunas ligeras diferencias.

Sin embargo, el `backend` también requiere saber un poco de todo.

Infraestructura, Cloud, DevOps, Bases de Datos, etc.